

# ROS (part II)

ROBOTICS



**POLITECNICO**  
MILANO 1863

# Schedule V. 2.0



|    |                                                         |     |                                                                                  |
|----|---------------------------------------------------------|-----|----------------------------------------------------------------------------------|
| L1 | Middleware for robotics and ROS<br>Installation Party   | L6  | parameters, message filter, rosbag tools,<br>actionlib                           |
| L2 | Ros workspace, publisher/subscriber                     | L7  | ROS on multiple machines, time<br>synchronization, latched pub, async<br>spinner |
| L3 | Publisher, subscriber, launch file , custom<br>messages | L8  | Robot Navigation, Stage, Gmapping                                                |
| L4 | Services, parameters, timers, node<br>architecture      | L9  | Robot Navigation (Part II), Robot<br>Localization, mapviz                        |
| L5 | TF, parameters, first project                           | L10 | ROS2, foxglove, second project                                                   |

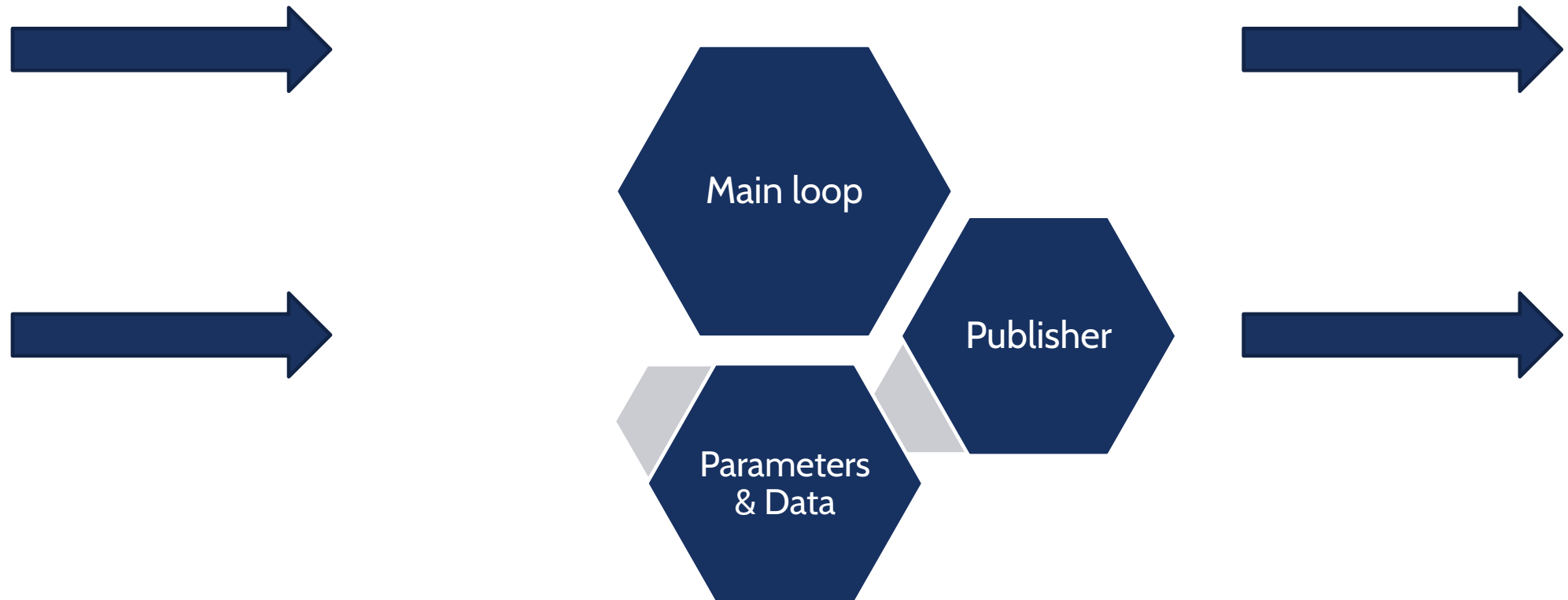
# PARAMETERS

ROBOTICS



**POLITECNICO**  
MILANO 1863

# INSIDE THE NODE





# USING PARAMETERS

## 2 ways to use parameters:

- Look at the value before entering main loop
- Add callback to parameters change

## 3 ways to set parameters:

- command line
- launch file
- `rqt_reconfigure`



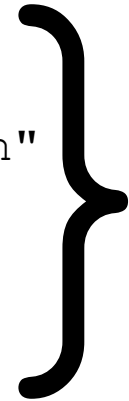
# PARAMETER CHECK BEFORE MAIN LOOP

Similar code to publisher/subscriber example:

```
#include "ros/ros.h"
```

```
#include "std_msgs/String.h"
```

```
#include <sstream>
```



Standard include



# PARAMETER CHECK BEFORE MAIN LOOP

Similar code to publisher/subscriber example:

```
int main(int argc, char **argv){
```

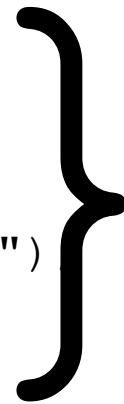
```
    ros::init(argc, argv, "param_first")
```

```
    ros::NodeHandle n;
```

```
    ros::Publisher chatter_pub = n.advertise<std_msgs::String>("parameter",  
1000);
```

```
    std::string name;
```

```
    (...)
```



ros initialization



Publisher creation



## PARAMETER CHECK BEFORE MAIN LOOP

Similar code to publisher/subscriber example:

`n.getParam("/name", name);` ← get parameter value

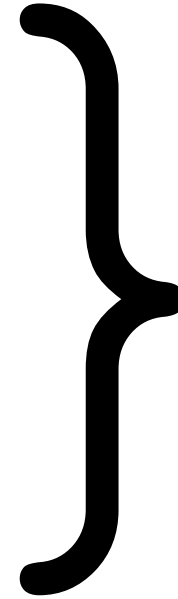
`ros::Rate loop_rate(10);` ← set loop rate



# PARAMETER CHECK BEFORE MAIN LOOP

Similar code to publisher/subscriber example:

```
while (ros::ok()) {  
    std_msgs::String msg;  
    msg.data = name;  
    ROS_INFO("%s", msg.data.c_str());  
    chatter_pub.publish(msg);  
    ros::spinOnce();  
    loop_rate.sleep();  
}
```



Main loop, not different from  
previous example



## PARAMETER CHECK BEFORE MAIN LOOP

Add the new file to CMakeLists.txt, as we did in pub/sub example

```
add_executable(param_first src/param_first.cpp)
target_link_libraries(param_first ${catkin_LIBRARIES})
```

Compile the new node



## PARAMETER CHECK BEFORE MAIN LOOP

Start the node:

- If no parameter is previously set the node will publish an empty string
- Set the parameter value using: “ rosparam set name "first" ”
- Now the node will publish “first” string
- If you change again the value while the node is running it will have no effect because the node looks at the value only once



## SETTING PARAMETER VALUE INSIDE THE LAUNCH FILE

A good practice with parameter is to set the value directly inside the launch file, so the user doesn't have to initialize the values using command line tools, add the line:

```
<param name="name" value="value" />
```

Inside a Launch file to set a parameter



# SETTING PARAMETER VALUE INSIDE THE LAUNCH FILE

Create a param\_set.launch file inside a launch folder

```
<launch>
```

```
<param name="name" value="second" /> ← Set the parameter value
```

```
<node pkg="parameter_test" name="param_first" type="param_first"
```

```
output="screen" ← Redirect the output of ROS_INFO to the terminal
```

```
/>
```

```
</launch>
```

## DYNAMIC RECONFIGURE



Previous examples allowed us to set the parameter value only once, to change the value while the node is running it's not recommended to insert the `getParam` call inside the mail loop because it's resource consuming and inefficient, to achieve this task we use dynamic reconfigure

# DYNAMIC RECONFIGURE



First create a cfg folder and inside a parameters.cfg file, than make it executable:

```
chmod +x parameters.cfg
```

Now we can start writing the configuration file; cfg file are not written in c++ but in python

# DYNAMIC RECONFIGURE



```
#!/usr/bin/env python
```

```
PACKAGE = "parameter_test"
```



Set the package of the node

```
from dynamic_reconfigure.parameter_generator_catkin import *
```



Import for dynamic reconfigure

```
gen = ParameterGenerator()
```



Create a generator



# DYNAMIC RECONFIGURE

To add a parameter we use the command:

```
gen.add ("name", type, level, "description", default, min, max)
```

In our case:

```
gen.add("int_param",    int_t,    0, "An Integer parameter", 50,  0, 100)
```

```
gen.add("double_param", double_t, 0, "A double parameter",   .5, 0,  1)
```

```
gen.add("str_param",    str_t,    0, "A string parameter",  "Hello World")
```

```
gen.add("bool_param",   bool_t,   0, "A Boolean parameter",  True)
```



## DYNAMIC RECONFIGURE

We can also create multiple choice parameter using enum, first create an enum using a list of const; to create a constant:

```
gen.const ("name", type, value, "description")
```

Then create the enum:

```
my_enum = gen.enum([const_1, const_2, ...], "description")
```

Last we add the enum like previously

```
gen.add ("name", type, level, "description", default, min, max, edit_method =  
my_enum)
```



# DYNAMIC RECONFIGURE

In our case we create a size parameter with four values:

```
size_enum = gen.enum([ gen.const("Small",      int_t, 0, "A small constant"),
                       gen.const("Medium",     int_t, 1, "A medium constant"),
                       gen.const("Large",      int_t, 2, "A large constant"),
                       gen.const("ExtraLarge", int_t, 3, "An extra large
constant") ],
                    "An enum to set size")

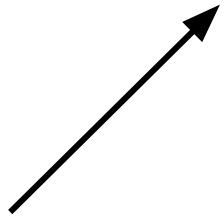
gen.add("size", int_t, 0, "A size parameter which is edited via an enum", 1,
0, 3, edit_method=size_enum)
```



## DYNAMIC RECONFIGURE

Lastly we have to tell the generator to generate the files:

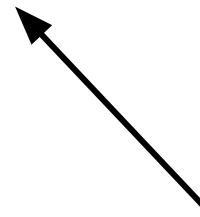
```
gen.generate("package name", "node_name", "prefix")
```



Name of the node



Name of the node



Name of the prefix

The prefix value is the string used to create the name of the header file you will have to include, with the name `prefixConfig.h`

# DYNAMIC RECONFIGURE



In our case we write:

```
exit(gen.generate(PACKAGE, "param_second", "parameters"))
```

Now we can write a node using those parameters, create a file  
“param\_second.cpp” in your src folder

# DYNAMIC RECONFIGURE



```
#include <ros/ros.h>
```

```
#include <dynamic_reconfigure/server.h>
```

} Standard include

```
#include <parameter_test/parametersConfig.h>
```



Include the previously  
generated file



# DYNAMIC RECONFIGURE

```
int main(int argc, char **argv) {
```

```
    ros::init(argc, argv, "param_second"); ← ROS initialization
```

```
    dynamic_reconfigure::Server<parameter_test::parametersConfig> server;
```



Create the parameter server specifying the type of config

```
    dynamic_reconfigure::Server<parameter_test::parametersConfig>::CallbackType f;
```



Create the callback

# DYNAMIC RECONFIGURE



```
f = boost::bind(&callback, _1, _2);
```

← Bind the callback

```
server.setCallback(f);
```

← Set the server callback

```
ROS_INFO("Spinning node");
```

```
ros::spin();
```

```
return 0;
```

}

keep spinning

# DYNAMIC RECONFIGURE



```
void callback(parameter_test::parametersConfig &config, uint32_t level) {
```



Create the callback

Pointer to the parameters structure



Value of the level bitmask



# DYNAMIC RECONFIGURE



Last we print all the parameters value

```
ROS_INFO("Reconfigure Request: %d %f %s %s %d",  
        config.int_param, config.double_param,  
        config.str_param.c_str(),  
        config.bool_param?"True":"False",  
        config.size);
```



## DYNAMIC RECONFIGURE

We also have to edit the CMakeLists.txt, to the find\_package call

add: “dynamic\_reconfigure”

Also add the .cfg file:

```
generate_dynamic_reconfigure_options(  
    cfg/parameters.cfg  
)
```

And to prevent to first create the header file and than compile our node use:

```
add_dependencies(param_second ${PROJECT_NAME}_gencfg)
```

## DYNAMIC RECONFIGURE



The level bitmask can be used to get what parameter has changed, edit the parameters.cfg file and set unique values to the level field

In the param\_second.cpp callback add:

```
ROS_INFO ("%d", level);
```

To print the index of the label of the level value of the changed parameter

# ACTIONLIB

ROBOTICS



**POLITECNICO**  
MILANO 1863

# WHAT IS ACTIONLIB



Node A sends a request to node B to perform some task

## **Service**

Small execution time

Requesting node can wait

No status

No cancellation

## **Action**

Long execution time

Requesting node cannot wait

Status monitoring

Cancellation

# WHAT IS ACTIONLIB



actionlib package is:

- sort of ROS implementation of threads

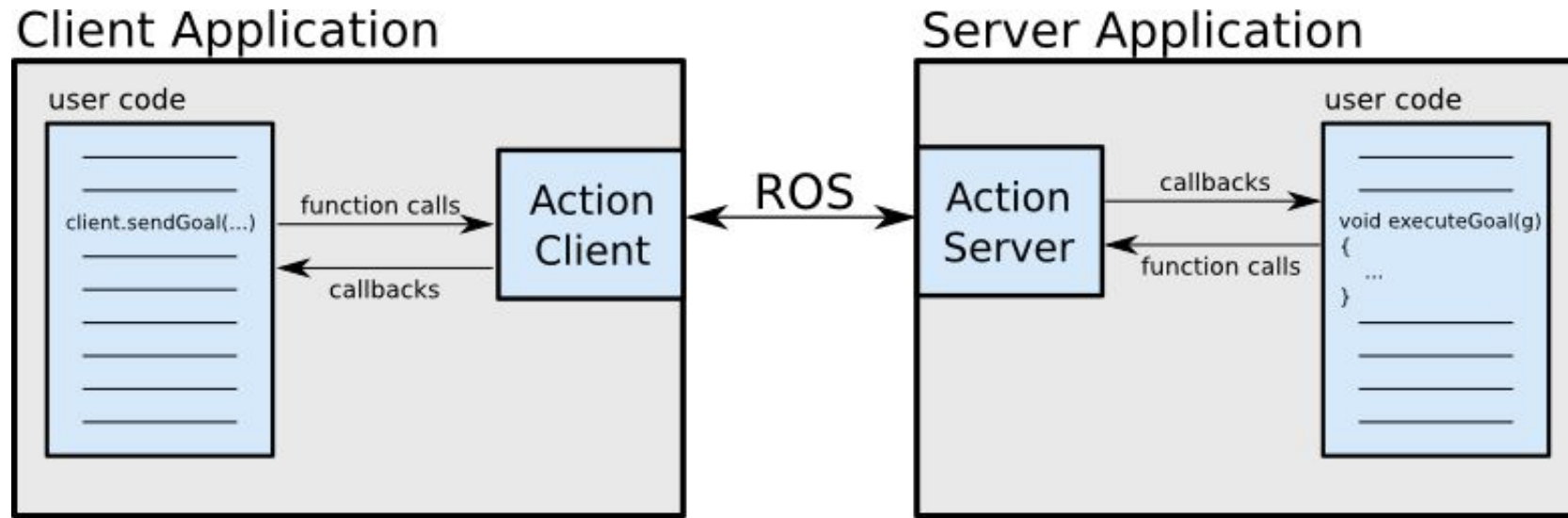
- based on a client/server paradigm

And provides tools to:

- create servers that execute long-running tasks (that can be preempted).

- create clients that interact with servers

# WHAT IS ACTIONLIB

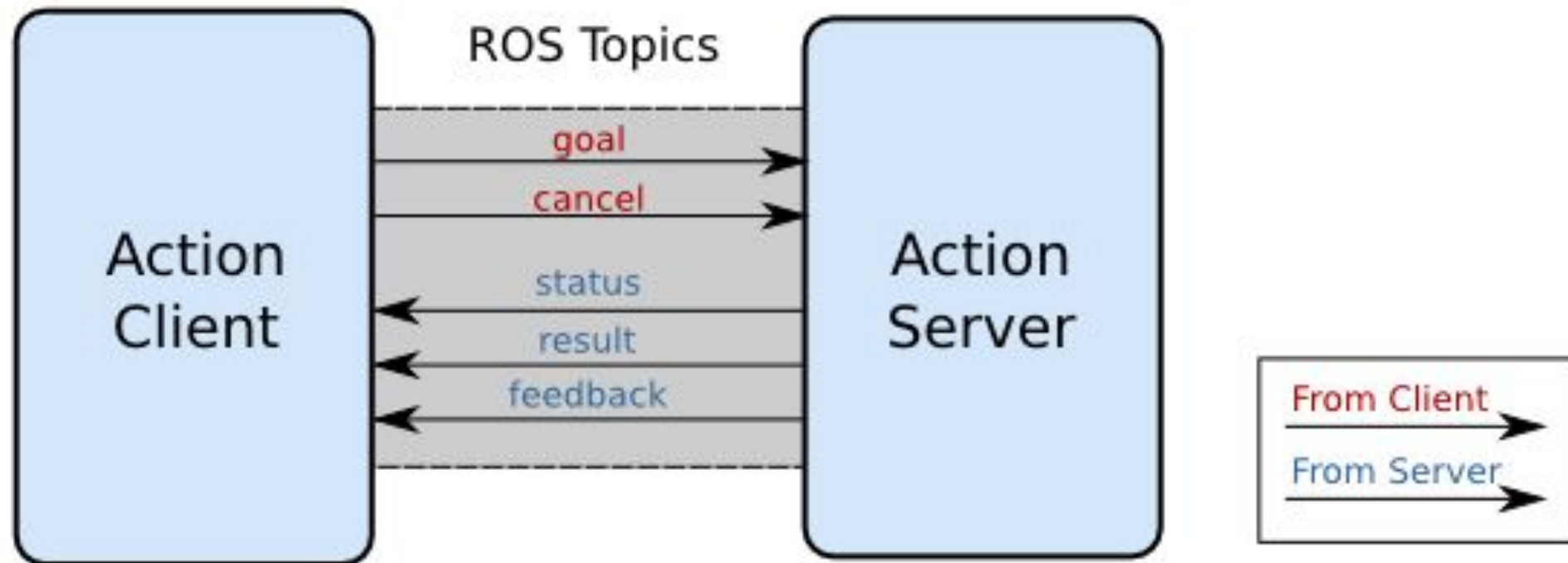


The ActionClient and ActionServer communicate via a "ROS Action Protocol", which is built on top of ROS messages

# CLIENT-SERVER INTERACTION



## Action Interface



# CLIENT-SERVER INTERACTION



**goal:** to send new goals to server

**cancel:** to send cancel requests to server

**status:** to notify clients on the current state of every goal in the system.

**feedback:** to send clients periodic auxiliary information for a goal

**result:** to send clients one-time auxiliary information upon completion of a goal

# ACTION AND GOAL ID

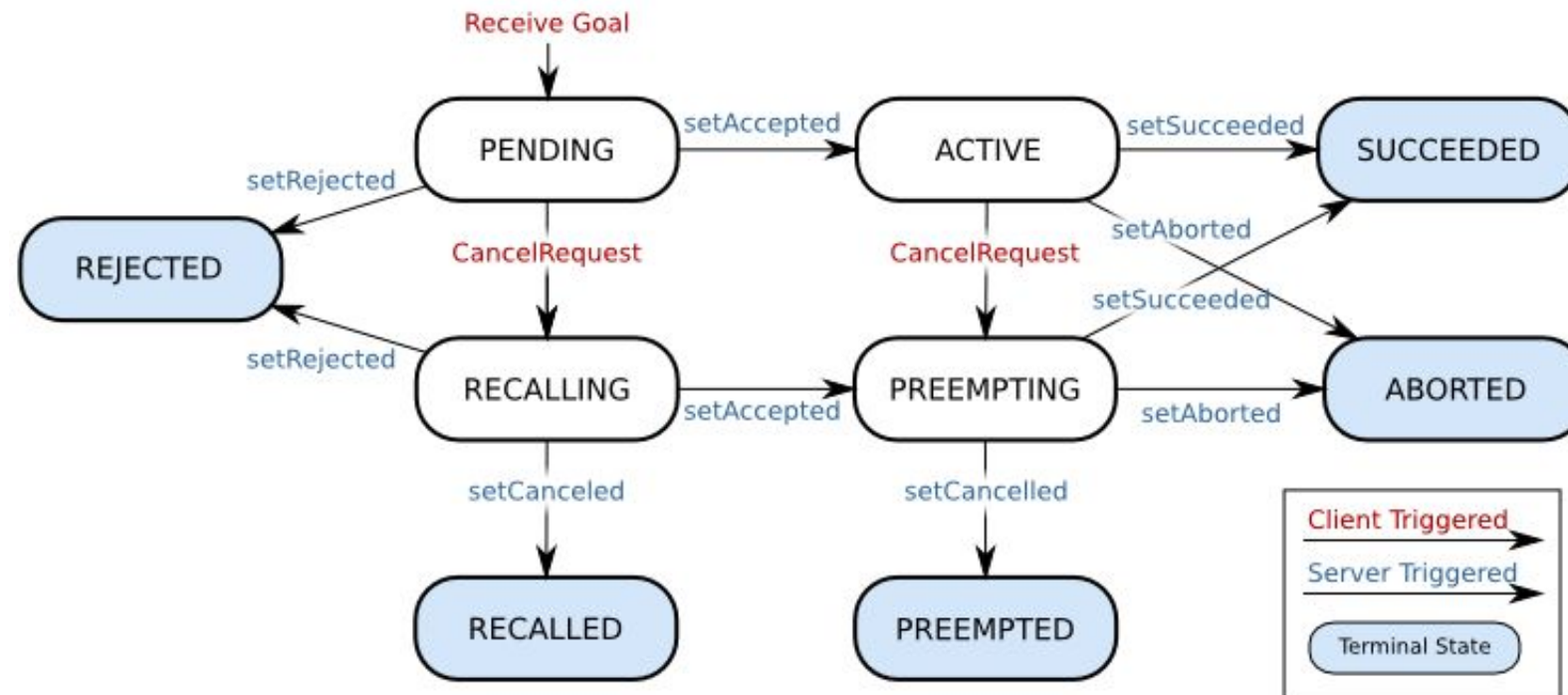


Action templates are defined by a name and some additional properties through an `.action` structure defined in ROS

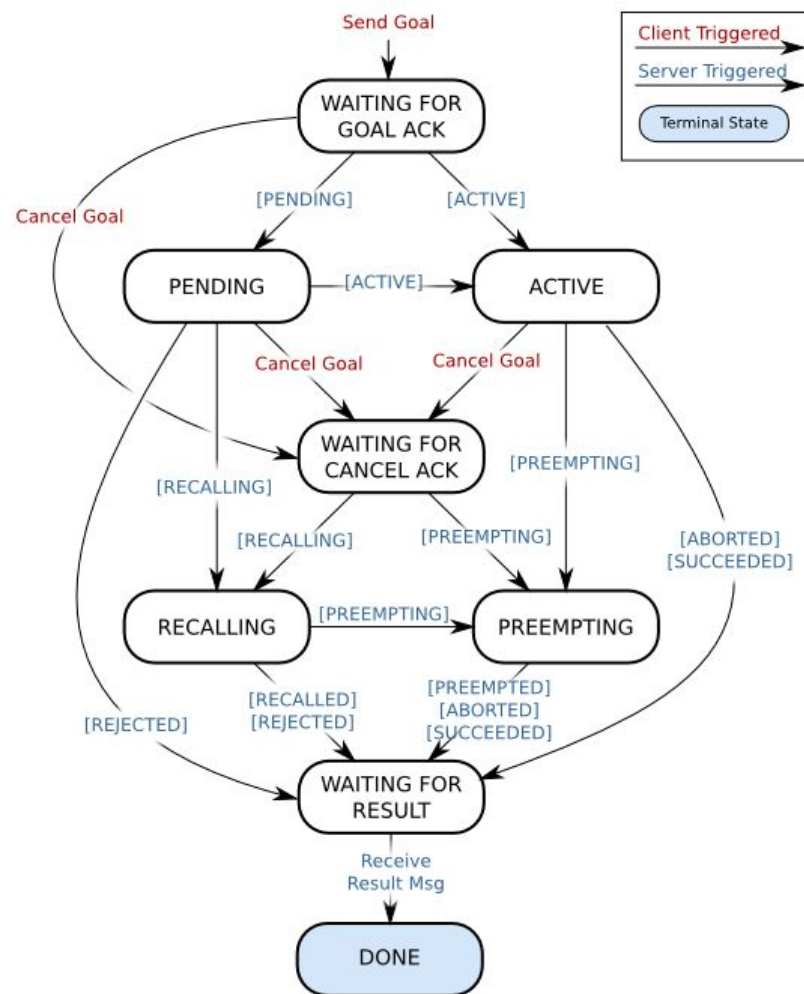
Each *instance* of an action has a unique Goal ID

Goal ID provides the action server and the action client with a robust way to monitor the execution of a particular instance of an action.

# SERVER STATE MACHINE



# CLIENT STATE MACHINE



## .ACTION EXAMPLE



# Define the goal

uint32 dishwasher\_id # Specify the dishwasher id

---

# Define the result

uint32 total\_dishes\_cleaned

---

# Define a feedback message

float32 percent\_complete

# SIMPLEACTIONSERVER



```
int main(int argc, char** argv) {  
    ros::init(argc, argv, "do_dishes_server");  
    ros::NodeHandle n;  
    Server server(n, "do_dishes", boost::bind(&exe, _1, &server), false);  
    server.start();  
    ros::spin();  
    return 0;  
}
```

# SIMPLEACTIONSERVER



```
void exe(const chores::DoDishesGoalConstPtr& goal, Server* as) {
    while(allClean()) {
        doDishes(goal->dishwasher_id)
        if(as->isPreemptRequested() || !ros::ok()) {
            as->setPreempted();
            break;
        }
        as->publishFeedback(currentWork(goal->dishwasher_id))
    }
    if(currentWork(goal->dishwasher_id) == 100)
        as->setSucceeded();
}
```

# SIMPLEACTIONCLIENT



```
#include <chores/DoDishesAction.h>
```

```
#include <actionlib/client/simple_action_client.h>
```

```
typedef actionlib::SimpleActionClient<chores::DoDishesAction> Client;
```

# SIMPLEACTIONCLIENT



```
int main(int argc, char** argv) {  
    ros::init(argc, argv, "do_dishes_client");  
    Client client("do_dishes", true); // true -> don't need ros::spin()  
    client.waitForServer();  
    chores::DoDishesGoal goal;  
    //set goal parameters  
    goal.dishwasher_id = pickDishwasher();  
}
```

# SIMPLEACTIONCLIENT



```
client.sendGoal(goal);
client.waitForResult(ros::Duration(5.0));
if (client.getState() == actionlib::SimpleClientGoalState::SUCCEEDED)
    ROS_INFO("Yay! The dishes are now clean");
std::string state = client.getState().toString();
ROS_INFO("Current State: %s\n", state.c_str());
return 0;
}
```

# TESTING



Copy the `actionlib_tutorial` folder inside the `src` folder of your catkin workspace and compile it

To start the server:

```
$ rosrun actionlib_tutorials fibonacci_server
```

The client has some parameters that can be set in the launch file, order and duration; after setting those parameters call:

```
$ roslaunch actionlib_tutorials launcher.launch
```

# TESTING



You can monitor the server status simply using topics:

```
$ rostopic list
```

To get the feedback from the server:

```
$ rostopic echo /fibonacci/feedback
```

# ROSBAG tools

ROBOTICS



**POLITECNICO**  
MILANO 1863

# ROSBAG tools



Perform operation directly on bags

Convert bags, extract data from bags, edit bags

python api/c++ api

# Bag to csv



```
import rosbag
```

```
import csv
```

← include for rosbag, csv, msg

```
from turtlesim.msg import Pose
```

```
bag = rosbag.Bag('/home/airlab/robotics/bags/pose.bag')
```

← Open the bag

```
f = open('/home/airlab/robotics/bags/pose.csv', 'w')
```

← Open the log file

```
writer = csv.writer(f)
```

← Create the csv writer

```
header = ['timestamp', 'x', 'y', 'theta' ]
```

```
writer.writerow(header)
```

← Write header

# Bag to csv



```
for topic, msg, t in bag.read_messages(topics=['/turtle1/pose']):  
    writer.writerow ([t,msg.x, msg.y,msg.theta])  
bag.close()  
f.close()
```

← Iterate on msg  
← Write to csv

# Bag editor



```
import rosbag
```

```
import csv
```

```
from turtlesim.msg import Pose
```

← include for rosbag, csv, msg

```
bag_in = rosbag.Bag('/home/airlab/robotics/bags/pose.bag') ← Open the input bag
```

```
bag_out = rosbag.Bag('/home/airlab/robotics/bags/pose_shifted.bag', 'w')
```

↑  
Open the output bag

# Bag editor



```
for topic, msg, t in bag_in.read_messages(topics=['/turtle1/pose']):  
    msg.x += 2  
    t.secs += 30  
    bag_out.write ('/turtleS/pose', msg, t)  
bag_in.close()  
bag_out.close()
```

← Iterate on msg

← Write to out bag

# MESSAGE FILTERS

ROBOTICS



**POLITECNICO**  
MILANO 1863

# MESSAGE FILTERS



Useful to synchronize multiple topics

Need topics with header and timestamp

Can synchronize with exact time or approximate time

Camera topics synchronization has a custom version



# MESSAGE FILTERS (without policy)

```
message_filters::Subscriber<geometry_msgs::Vector3Stamped> sub1(n, "topic1",  
1);
```

← **Create the subscriber**

```
message_filters::Subscriber<geometry_msgs::Vector3Stamped> sub2(n, "topic2",  
1);
```

```
message_filters::TimeSynchronizer<geometry_msgs::Vector3Stamped,  
geometry_msgs::Vector3Stamped> sync(sub1, sub2, 10);
```

```
sync.registerCallback(boost::bind(&callback, _1, _2));
```

↑  
**Bind it with the callback**

↑  
**Create the time synchronizer**



# MESSAGE FILTERS (with policy)

```
typedef  
message_filters::sync_policies::ExactTime<geometry_msgs::Vector3Stamped,  
geometry_msgs::Vector3Stamped> MySyncPolicy;
```



Create the policy

```
message_filters::Synchronizer<MySyncPolicy> sync(MySyncPolicy(10), sub1, sub2);  
sync.registerCallback(boost::bind(&callback, _1, _2));
```



Bind it with the callback



Create the time synchronizer with  
the policy